

Stress Test #3

Contents

Stress Test #3 — Stuff8 Scaling Limits	1
Setup	1
Results: Internal Gateway	1
Results: Cloudflare	2
Comparison: Stuff8 vs Assessment	2
Throughput Ceiling	2
Cloudflare Ceiling	3
TL;DR	3

Stress Test #3 — Stuff8 Scaling Limits

Date: 7 August 2026 **Target:** stuff8.com **Previous:** Stress Test #2 — Assessment

Setup

4 cloudflared replicas deployed for the stuff8 tunnel (1 original + 3 replicas via `cloudflared-stuff8@service`). Same systemd template pattern as the jogjaitcamp tunnel from Stress Test #1.

The stuff8 tunnel routes 9 hostnames through CT 101 (192.168.88.30):
- stuff8.com, chat.stuff8.com, photos.stuff8.com - eco.stuff8.com, eco-sphere.stuff8.com - hooks-*.stuff8.com (deploy webhooks)

Results: Internal Gateway

VUs	Total Re-quests	Avg	Median	P95	Fail-ures	Req/s	MB/s
1,000	166,195	288ms	161ms	194ms	0%	2,767	45.8
2,000	153,473	608ms	536ms	643ms	0%	2,551	42.2
3,000	147,749	959ms	868ms	1,091ms	0%	1,848	30.6
4,000	141,530	1,329ms	1,272ms	1,529ms	0.2%	1,770	29.2
5,000	142,588	1,601ms	1,721ms	2,095ms	0%	1,920	31.7

Results: Cloudflare

VUs	Total Requests	Avg	Median	P95	Failures	Req/s	MB/s
1,000	4,082	12,263ms	13,759ms	16,435ms	0.1%	49	0.8
2,000	6,377	8,644ms	12,236ms	15,500ms	32.3%	73	0.8
3,000	8,473	6,259ms	2,329ms	14,280ms	49.2%	98	0.9

Comparison: Stuff8 vs Assessment

VUs	Stuff8 Avg	Assessment Avg	Delta
1,000	288ms	403ms	Stuff8 29% faster
2,000	608ms	788ms	Stuff8 23% faster
3,000	959ms	1,204ms	Stuff8 20% faster
4,000	1,329ms	1,590ms	Stuff8 16% faster
5,000	1,601ms	2,005ms	Stuff8 20% faster

Stuff8 consistently outperforms assessment by 20–29% across all load levels. This is expected — stuff8’s frontend is Astro (static generation where possible) and its backends are Rust (lightweight), while assessment runs a larger SvelteKit frontend + Java Spring Boot backends with JPA/Hibernate overhead.

Both estates share CT 101 (10 cores, 6 GiB RAM) with apindo, eco_docs, and ecosphere. The Rust-based stuff8 services leave more CPU headroom than the JVM-based assessment services.

Throughput Ceiling

Both estates converge on similar throughput ceilings:

Estate	Max Req/s	Max MB/s	Primary Tech Stack
Stuff8	2,767	45.8	Astro + Rust (Axum)
Assessment	1,982	32.8	SvelteKit + Spring Boot (Java)

The shared CT 101 infrastructure (Caddy gateway, Proxmox bridge, NIC) imposes an upper bound of ~2,500–2,800 req/s for both estates combined. The individual estate ceiling depends on how CPU-intensive each request is — Rust backends serve more req/s per CPU cycle than Java backends.

Cloudflare Ceiling

Identical pattern to assessment: ~100 req/s cap through Cloudflare regardless of replicas. At 4 replicas per tunnel (jogjaitcamp + stuff8 = 8 cloudflared processes total in CT 100), the proxy CT's 4 cores and the office ISP upload remain the limiting factors.

TL;DR

Stuff8 handles 5,000 concurrent users with zero failures and 1,601ms average. It is 20–29% faster than assessment at every load level thanks to its lighter tech stack (Astro + Rust vs SvelteKit + Java). Both estates share the same Cloudflare tunnel ceiling of ~100 req/s, which is sufficient for current usage but would need a VPS edge node for higher public traffic.